Automated Insurance Claim Triage

Building an AI Prompt Chain with Pydantic Validation

Lesson 4: Chaining Prompts for Agentic Reasoning · GreatLearning Agentic AI

01 EXTRACT › 02 ASSESS › 03 ROUTE

The Challenge: Processing Raw Accident Reports

✗ The Old Way — Manual Processing

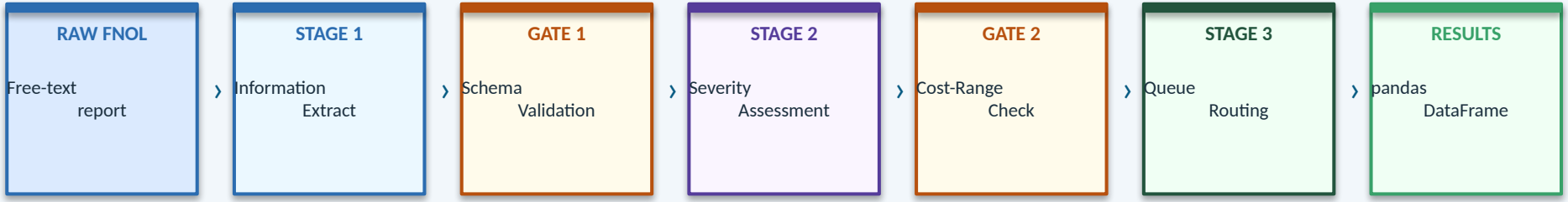
- Human agent reads each accident report
- Manually finds: claim ID, customer, vehicle, damaged parts
- Judges damage severity based on experience
- Decides which specialist team should handle it
- Slow — one report at a time
- Inconsistent — different agents, different conclusions

VS

✓ The New Way — AI Prompt Chain

- 3 specialised AI prompts chain together automatically
- Stage 1 extracts structured data from free text
- Stage 2 assesses damage severity with cost estimate
- Stage 3 routes to the correct specialist queue
- Gate checks validate output at every single step
- Processes 5+ reports in seconds, consistently

The 3-Stage AI Pipeline — Big Picture



Stage 1 — Extract: Reads raw FNOL text → ClaimInformation (claim_id, name, vehicle, loss_desc, damage_area)

Stage 2 — Assess: Structured claim → SeverityAssessment (Minor/Moderate/Major + estimated repair cost in $)

Stage 3 — Route: Claim + severity → ClaimRouting (queue: glass/fast_track/material_damage/total_loss + priority 1–5)

Gate Checks: After every AI call, Pydantic validates the JSON output. Bad data is rejected before reaching the next stage.

The Raw Input: 5 First Notice of Loss (FNOL) Reports

Each report is a plain text string — unstructured, informal, typed by the customer

C001

John Smith

2018 Toyota Camry

Incident:

Rock hit windshield causing a small chip about the size of a quarter. No other damage observed.

C002

Sarah Johnson

2020 Honda Civic

Incident:

Parked at grocery store. Someone hit my car and dented the rear bumper and taillight. Taillight broken.

C003

M. Rodriguez

2022 Ford F-150

Incident:

Serious collision at intersection. Airbags deployed. Hood, bumper, engine damaged. Vehicle not drivable.

C004

Emma Williams

2019 Subaru Outback

Incident:

Hailstorm — multiple dents on hood, roof, trunk. Side mirrors damaged and one window has a small crack.

C005

David Brown

2021 Tesla Model 3

Incident:

Keyed in parking lot. Deep scratches along both driver's side doors.

`Python variable: sample_fnols → list[str] → each processed by extract_claim_info(fnol_text)`

Foundation: Libraries, AI Client & Shared Helper Function

Libraries & AI Client Setup

```
from openai  import OpenAI
from pydantic import BaseModel, Field
from typing   import List, Literal, Optional
import json

# Connect to the AI service once
client = OpenAI(
  base_url="https://openai.vocareum.com/v1",
  api_key="your-api-key"
)

# Shared model used by all 3 stages
MODEL = "gpt-4.1-nano"
```

Message list structure sent to every AI call:

```
{ "role":"system", "content": system_prompt }
{ "role":"user",   "content": claim_data }
```

get_completion() — Shared AI Caller

```
def get_completion(
    messages,          # list of role/content dicts
    system_prompt,     # prepended as "system"
    user_prompt,       # appended as "user"
    model = MODEL      # default: gpt-4.1-nano
) -> str:              # returns raw AI text

    response = client.chat.completions.create(
        model    = model,
        messages = messages,
        temperature = 0.7
    )
    return response.choices[0].message.content
```

Called by all 3 stages. Returns the raw text the AI wrote — e.g. a JSON string like {"severity":"Minor","est_cost":250.0} — which the gate then parses and validates.

Stage 1: Information Extraction — Overview

Input

- Raw FNOL text string
- Free-form, informal language
- Contains: Claim ID, name, vehicle, incident
- No structure — just a paragraph
- Variable: sample_fnols[i]

```
extract_claim_info(
  fnol_text: str
)
```

›

AI Does

- Reads the raw text report
- Identifies and extracts claim_id
- Extracts customer name
- Identifies vehicle details
- Tags damage_area from allowed list
- Returns JSON only — nothing else

```
get_completion(
  messages=[system, user]
) -> JSON str
```

›

Output

- Validated ClaimInformation object
- claim_id: "C001"
- name: "John Smith"
- vehicle: "2018 Toyota Camry"
- loss_desc: "Rock hit windshield..."
- damage_area: ["windshield"]

```
ClaimInformation(
  claim_id, name,
  vehicle, loss_desc,
  damage_area=[...]
)
```

Stage 1: Code Walkthrough — extract_claim_info()

① Function Entry Point

```
def extract_claim_info(
    fnol_text          # str: raw FNOL report
) -> Optional[ClaimInformation]:

    messages = [
      {"role":"system",
       "content": info_extraction_system_prompt},
      {"role":"user",
       "content": fnol_text}  # raw text input
    ]
    response = get_completion(messages=messages)
```

② Prompt Engineering — System Prompt

Role: "You are an auto insurance claim processing assistant"

Format: Return ONLY a valid JSON object

Fields: claim_id, name, vehicle, loss_desc, damage_area

Constraint: damage_area must use only 12 pre-defined values

Key rule: "Only respond with the JSON object, nothing else."

③ Pydantic Model — Validates AI Output

```
class ClaimInformation(BaseModel):
  claim_id:    str = Field(min_length=2,
                           max_length=10)
  name:        str = Field(min_length=2,
                           max_length=100)
  vehicle:     str = Field(min_length=2)
  loss_desc:   str = Field(min_length=10)
  damage_area: List[Literal[
    "windshield","front","rear","side",
    "roof","hood","door","bumper","trunk",
    "glass","fender","quarter panel"]]
```

④ Gate 1 — gate1_validate_claim_info()

```
def gate1_validate_claim_info(
    claim_info_json: str  # raw AI response text
) -> ClaimInformation:   # or raises ValueError

    # Step 1: text string → Python dict
    d = json.loads(claim_info_json)

    # Step 2: dict → Pydantic (validates all fields)
    return ClaimInformation(**d)

    # ValueError if any field fails → caller returns None
```

Stage 2: Severity Assessment — Overview

Input

- ClaimInformation object from Stage 1
- Converted via .model_dump_json()
- Becomes the "user" message to AI
- Contains all structured claim fields

```
assess_severity(
  claim_info: ClaimInformation
)
```

›

AI Does

- Reads the structured claim JSON
- Applies carrier heuristics:
  - › → Minor: chips/scratches ($100–$1k)
  - › → Moderate: panel damage ($1k–$5k)
  - › → Major: structural ($5k–$50k)
- Returns severity + cost as JSON

```
get_completion(
  messages=[system, user]
) -> JSON str
```

›

Output

- Validated SeverityAssessment
- C001: Minor, $250
- C002: Moderate, $2,500
- C003: Major, $25,000
- C004: Moderate, $3,000
- C005: Moderate, $2,500

```
SeverityAssessment(
  severity: "Minor"|
    "Moderate"|"Major",
  est_cost: float > 0
)
```

Stage 2: Code Walkthrough — assess_severity()

① Function Entry Point

```
def assess_severity(
    claim_info: ClaimInformation  # from Stage 1
) -> Optional[SeverityAssessment]:

    # Convert Pydantic object → JSON text
    claim_info_json = claim_info.model_dump_json()

    messages = [
      {"role":"system",
       "content": severity_assessment_system_prompt},
      {"role":"user",
       "content": claim_info_json}  # Stage 1 output
```

② Prompt Engineering — System Prompt

Role: "You are an auto insurance damage assessor"

Rules: Carrier heuristics embedded: Minor $100-$1k, Moderate $1k-$5k, Major $5k-$50k

Output: JSON with only severity and est_cost

Effect: AI knows expected cost ranges before answering — improves Gate 2 pass rate

③ Pydantic Model — Validates AI Output

```
class SeverityAssessment(BaseModel):
  severity: Literal[
    "Minor",    # $100 - $1,000
    "Moderate", # $1,000 - $5,000
    "Major"     # $5,000 - $50,000
  ]
  est_cost: float = Field(gt=0)  # > 0

# Rejects: "moderate" (wrong case)
# Rejects: -500.0   (negative value)
```

④ Gate 2 — gate2_cost_range_ok() [Business Logic]

```
def gate2_cost_range_ok(severity_json):
    d = json.loads(severity_json)
    v = SeverityAssessment(**d)  # Pydantic first

    # Extra range check (pure Python if-statements):
    if v.severity=="Minor" and (
          v.est_cost<100 or v.est_cost>1000):
        raise ValueError("Cost out of Minor range")
    elif v.severity=="Moderate" and (...): ...
    elif v.severity=="Major"    and (...): ...
```

Stage 3: Queue Routing — Overview

Input

- ClaimInformation + SeverityAssessment
- Both merged into one combined dict
- json.dumps() → single JSON string
- Guard: if severity is None → skip

```
route_claim(
  claim_info: ClaimInformation,
  severity: SeverityAssessment
)
```

›

AI Does

- glass → Minor + only glass damage
- fast_track → Other Minor damage
- material_damage → All Moderate
- total_loss → All Major
- Priority 1 = safety/stranded
- Priority 5 = cosmetic only

```
get_completion(
  messages=[system, user]
) -> JSON str
```

›

Output

- Validated ClaimRouting object
- C001: glass, p=4
- C002: material_damage, p=3
- C003: total_loss, p=2
- C004: material_damage, p=3
- C005: material_damage, p=3

```
ClaimRouting(
  claim_id: str,
  queue: Literal[...],
  priority: int (1-5)
)
```

Stage 3: Code Walkthrough — route_claim()

① Function Entry Point

```
def route_claim(
    claim_info: ClaimInformation,
    severity_assessment: Optional[SeverityAssessment]
) -> Optional[ClaimRouting]:

    if severity_assessment is None: return None

    routing_input = {
      "claim_info":          claim_info.model_dump(),
      "severity_assessment": severity.model_dump()
    }  # Both Stage 1 + Stage 2 outputs combined
```

② Prompt Engineering — System Prompt

Role: "You are an auto insurance claim routing specialist"

Rules: 4 routing queues explained with conditions

Priority: Scale 1-5 defined (1=safety/stranded, 5=cosmetic)

User msg: json.dumps(routing_input) — full combined JSON from Stage 1+2

③ Pydantic Model — Validates AI Output

```
class ClaimRouting(BaseModel):
  claim_id: str
  queue: Literal[
    "glass",           # Minor + glass only
    "fast_track",      # Other minor
    "material_damage", # Moderate
    "total_loss"       # Major
  ]
  priority: int = Field(
    ge=1,  # >= 1  (1 = most urgent)
    le=5   # <= 5  (5 = lowest)
```

④ Gate 3 — gate3_validate_routing()

```
def gate3_validate_routing(
    routing_json: str    # raw AI text
) -> ClaimRouting:

    d = json.loads(routing_json)  # text → dict
    return ClaimRouting(**d)      # Pydantic checks

    # Rejects: queue="fast track" (space = invalid)
    # Rejects: priority=6         (max is 5)
    # Rejects: priority="high"    (must be int)
```

Final Results — All 5 Claims Triaged Successfully

`Stored in routed_claim_items → merged into a pandas DataFrame`

Claim	Customer	Damage Areas	Severity	Est. Cost	Queue	Priority
C001	**John Smith** *2018 Toyota Camry*	windshield	Minor	$250	glass	4
C002	**Sarah Johnson** *2020 Honda Civic*	bumper, rear	Moderate	$2,500	material_damage	3
C003	**M. Rodriguez** *2022 Ford F-150*	front, hood, bumper	Major	$25,000	total_loss	2
C004	**Emma Williams** *2019 Subaru Outback*	hood, roof, trunk, side, glass	Moderate	$3,000	material_damage	3
C005	**David Brown** *2021 Tesla Model 3*	door	Moderate	$2,500	material_damage	3

Key Concepts Learned in This Notebook

Prompt Chaining

Instead of one giant prompt, the task is split into 3 specialised AI calls — each with a single, focused job. The output of each step becomes the structured input for the next.

Gate Checks

After every AI call, Python code validates the output before passing it forward. This prevents errors from silently cascading — bad data is caught immediately at the source.

Pydantic Validation

Pydantic models define exactly what fields, types, and values are allowed. Any AI response that does not match the schema is automatically rejected, with a clear error message.

Agentic Reasoning

The system reasons step-by-step like a human expert: understand → judge → decide. Each stage builds on the last, and business rules are encoded directly into the prompts.

What Did We Achieve?

Five raw accident reports went in → three chained AI calls transformed them into fully validated, structured routing decisions — all without a human reading a single report manually.

5
FNOL Reports Processed

3
AI Prompt Stages

3
Gate Checks Passed

4
Output Queues

The same pattern applies to any domain requiring unstructured-text classification and routing:

- Medical Triage
- Legal Document Sorting
- Customer Support Tickets
- HR Application Review